

Weltwerkzeug — eine Fläche, drei Speicher

Stand: gebaut bis Schritt 4 plus Tageszeit, 20. September 2026.

`/editor` bleibt als Fach Prüfen ohne Partie. Schritt 9 (Anfassen auf der Bühne) ist geplant, noch nicht gebaut.

Ersetzt als Zielbild: `docs/SPIELLEITER.md` + `docs/EDITOR.md` .

Beide Oberflächen tun dasselbe: die Welt anfassen. Sie sollen **eine** werden, **während des Spiels** erreichbar.

Werkstatt — ehrlich

- Wissen: `KNOWLEDGE_META` , 21 Schlüssel, fertig.
- Dead-Node Fluss: echt, nur Übersicht `FLUSS` .
- Dead-Node Lager: **nicht mehr** zwei identische Listen. Liest `LAGER_WEGE` .
- Questpfad: echt, nur `QUEST_PFADE` .
- Asset-Check: HEAD, echt.
- Export: JSON + Zod. Import: Zod, kein Kanon-Schreiben.
- Zustand: **nicht** im Einsatz, aus den Dependencies.
- Monaco: nur Fach Prüfen, JSON, lazy. Nicht in den Kartentexten.

Warum das heute doppelt ist

Es gibt drei Werkzeuge und vier Speicher für denselben Vorgang.

Oberfläche	Wann	Was sie ändert
Titel → Texte im Spiel	Flag, dann inline auf der Karte	nur Zeilen
HUD → Spielleiter	während der Partie	Karte, Bild, Zustände, Lagen, Held
Titel → Werkstatt <code>/editor</code>	ohne Partie	Fluss, Probe, Wissen, Pfade, tote Knoten, Bilder, JSON

Speicher	Schlüssel	Inhalt
<code>lindendorf-save-v1</code>	eine Partie	Held (Leben, Flags, Log)
<code>lindendorf.spielleiter.karten.v1</code>	Hash der aktuellen Karte	Text, Bild, Karten-Zustände
<code>lindendorf-text-pack-v1</code> + Datei <code>text-pack.json</code>	Hash der Original -Karte	nur Titel, Zeilen, Wahlen
<code>lindendorf-author-v1</code> / <code>lindendorf.spielleiter.an</code>	an/aus	zwei Schalter für denselben Modus

Beim Speichern einer Karte schreibt das Spiel **beide** Patch-Speicher. Die Hashes sterben, sobald ein Quelltext sich ändert. Die Werkstatt kennt die Partie nicht. Der Spielleiter kennt Fluss und Probe nicht.

Das ist ein Werkzeug, das sich in drei Türen versteckt hat.

Zielbild

Ein **Weltwerkzeug**. Eine Schublade. Dieselbe Komponente mit und ohne laufende Partie.

- Im Spiel: HUD-Knopf **Welt** (ersetzt Spielleiter). `Alt+S` . `? welt=1` .
- Am Titel: **Weltwerkzeug** (ersetzt Werkstatt **und** den Extra-Schalter Textmodus).
- `/editor` fällt weg oder leitet nur noch auf dasselbe Blatt.
- Kein zweites State-Framework, kein Monaco, kein Git-Push aus der Fläche.

Die Partie bleibt sichtbar darunter. Die Schublade verdeckt den Text nicht dauerhaft; sie klappt.

Drei Speicher, ausdrücklich, nie gemischt

Jede Änderung sagt, **wohin** sie geht. Heute passiert das still.

1. Partie — der Held

Wohin: bestehendes `lindendorf-save-v1` (Knopf Speichern in der HUD).

Was: LP, Gold, Inventar, Flags, `effekte`, `entscheidungen`.

Wann: Gunst/Last am Held, Lage vorlegen, Wissen-Flags **nur** wenn eine Partie läuft und du „auf diesen Held“ wählst.

Lebensdauer: mit dem Spielstand. Neues Abenteuer = weg.
Anderer Browser = weg.

Ruf und Erinnerung sind hier keine eigenen Felder, sondern Ansichten auf `entscheidungen`. Eine Gunst-Aktion im Weltwerkzeug schreibt keinen Ruf-Wert direkt, sondern denselben Eintrag `{ typ: "ruf", ziel, wert }` ins Log, den auch eine Szene schreiben würde. `rufAus(held, ziel)` liest denselben Weg. Kein Sonderpfad am Spielcode vorbei — jede Werkzeug-Aktion ist eine simulierte Spielhandlung.

2. Auflage — die Welt in diesem Browser

Wohin: ein Speicher `lindendorf.welt.v2`. Ersetzt Karten-Store und Text-Pack-localStorage.

Was: Abweichung **pro Szene:** Titel, Zeilen, Wahlen (gleiche Anzahl), Bild, Portrait, Karten-Zustände an/fort.

Schlüssel: `szenaId` an `present({ id })`. Solange eine Szene keine Id hat: alter Text-Hash als Fallback, sichtbar als „hält nur, solange der Satz gleich bleibt“.

Wann: „Auf dieser Karte merken“. Auto-merken wie heute ist erlaubt, aber mit sichtbarer Marke *gemerkt* und einer Liste aller Auflagen.

Lebensdauer: über Partien hinweg, nur in diesem Browser. Quelltext-Änderung mit stabiler Id überlebt. Ohne Id stirbt die Auflage.

Verlauf statt nur Endstand: Jede Auflage speichert

`vorherigerText` (eine Ebene zurück, kein voller Undo-Stack).

Rückgängig stellt genau die letzte Fassung wieder her. Bewusst keine History wie in einem Texteditor – nur Schutz gegen versehentliches Überschreiben.

3. Kanon – der Quelltext des Spiels

Wohin: JSON-Module und `commitPack(..., apply=true)` in `quest-*.ts` / `content.ts` / `lager-content.ts`.

Was: Texte, die **jede** Partie so sehen soll.

Wann: nur nach Bestätigung **In den Kanon**. Nie still. Nie Git.

Lebensdauer: im Repo, sobald jemand committet. Bis dahin nur auf der Maschine, die den Dev-Server hat.

Bilder, die du hochlädst, liegen unter `public/art/sl/`. Die Auflage speichert den Pfad. In den Kanon kommen sie erst, wenn du sie einer `ArtKey` / `PortraitKey` zuweist – das bleibt Auftrag, keine Automatik.

Kanon-Diff vor dem Schreiben: „In den Kanon“ zeigt vorher eine zeilengenaue Gegenüberstellung Kanon gegen Auflage (nicht nur „n Karten weichen ab“). Ohne das committet man leicht eine Testzeile mit. Der Diff ist reine Anzeige, kein eigener Speicher.

Eine Fläche, drei Fächer

Nicht sieben Werkstatt-Reiter plus fünf Spielleiter-Akkordeons. Drei Fächer. Der Rest hängt darunter.

Fach Karte (Standard im Spiel)

Die aktuelle Szene. Genau das, was der Spielleiter schon kann, plus:

- Titel, Zeilen, Wahlen (Anzahl fest)
- Hintergrund, Portrait, Upload
- Zustände, die diese Karte **gibt** und **nimmt**
- Zurücksetzen auf den Kanon dieser Id
- **Rückgängig** (letzte Auflage-Fassung)
- **Diff-Ansicht** (Kanon vs. Auflage, ausklappbar, standardmäßig zu)
- Badge **gemerkt**, wenn diese Karte eine Auflage hat — auch sichtbar, wenn das Fach zu ist

Ohne laufende Szene: leer, Hinweis „erst spielen, dann die Karte anfassen“.

Speichert nach **Auflage** (2), nicht in den Held.

Fach Held

Nur mit laufender Partie, sonst Simulator mit Banner **nicht die Partie**.

- Gunst/Last auf den lebendigen Held → schreibt einen **entscheidungen**-Eintrag → Speicher **Partie** (1)
- Lage vorlegen (Herkunft) → Partie
- **Ruf** über **rufAus()**, nicht als Zahl editierbar — nur über Gunst/Last, damit jede Änderung im Log bleibt
- **Wissen**: **deriveKnowledge(held)**, gruppiert nach **KNOWLEDGE_META**-Typ (material / sozial / ort / übernatürlich)
- **Erinnerung**: Log-Einträge **typ=npv**, gruppiert nach Figur

- Probe **auf diesen Held** nur hinter einem extra Knopf; Standard-Probe schreibt nichts

Wissensflags aus der alten Werkstatt liegen hier. Kippen ohne „auf diesen Held“ bleibt Simulation und verfällt beim Schließen.

Fach Prüfen

Alles, was die alte Werkstatt konnte und **keine** Welt schreibt, außer du exportierst bewusst.

- Fluss durchklicken (Vorschau, startet keine Partie)
- Questpfade legen (Journal-Vorschau)
- Tote Knoten, Bildschlüssel
- **Schwierigkeitskurve**: Proben nach Ort gruppiert, min / max / Schnitt
- Module: JSON holen / lesen, **pro Modul einzeln** (Wissen, Proben, Wege, Quests), jeweils gegen das eigene Zodi-Schema
- **In den Kanon** nur hier, mit Zähler **und** Diff, bevor bestätigt wird
- Verweis auf den bestehenden Mobile-Smoke-Test statt eines zweiten Testwegs

Playwright bleibt außerhalb. Prüfen misst Fluss, Bilder, Wissen im Browser.

Öffnen während des Spiels – HUD

```
HUD:  Status ▾   Speichern   Wissen   Welt
```

Erweiterung um Zustandsanzeigen, nicht um extra Knöpfe:

Status ▾ Speichern Wissen (7) Welt • 3

- **Wissen (7)** — Größe von `deriveKnowledge(held)` . Kein neuer Zustand.
- **Welt • 3** — Punkt nur, wenn die **aktuelle** Szene eine Auflage hat. Die Zahl zählt alle abweichenden Karten in diesem Browser. Ohne Abweichung: schlicht „Welt“, kein leerer Platzhalter.
- Simulator-Banner (Held ohne Partie): feste Warnfarbe, nicht wie ein Fehler.
- Mobil: dieselbe Zeile, Schublade als **Bottom-Sheet** mit Safe-Area unten.
- Tasten: `Alt+S` öffnet Welt, `Esc` schließt die Schublade, Partie läuft weiter. Beides in `docs/PROJEKTKONTEXT.md` festhalten.

Welt öffnet Fach **Karte**.

Titel-Schalter Textmodus entfällt. Inline-Stifte auf der Bühne nur bei offener Schublade — sonst spielt man.

Kein zweites Panel. Kein `/editor` parallel zur Partie.

Was sich an der Engine ändern muss, bevor Speichern hält

Ohne das bleibt jede Auflage ein Würfelwurf gegen den nächsten Textschliff.

1. `present()` bekommt optionales `id: string` (`"lager-hub"` , `"muehle-eingang"`). Fehlt es, Fallback-Hash, in der UI als brüchig markiert.

2. Runtime reicht `id` in `SceneView`.
3. Lookup: erst Id, dann Hash, dann Titel-Notnagel.
4. Schema `WeltAuflage` (Zod), Version 2. Beim Laden: v1-Karten und Text-Pack **einmal** mergen, nicht dauerhaft doppeltschreiben.
5. Dieselbe Zod-Disziplin auf `entscheidungen` im Held (liegt schon in `heldSchema.ts`) – das Werkzeug darf sich nicht strenger prüfen als das Spiel.

Das ist der einzige Code-Schritt vor der Oberflächen-Zusammenlegung. Alles andere ist UI-Umzug.

Was ausdrücklich nicht in dieses Werkzeug gehört

- Wahl-Anzahl, Verzweigungen, Flags der Quests umbauen
 - Entscheidungs-Log von Hand fälschen (kommt über Spielhandlungen)
 - Neue ArtKeys ohne Bildplan
 - Ereignis-Würfel / Wetter – weiter leer, bis Auftrag
 - Automatisch Git, automatisch Kanon
 - Zustand-Library. Monaco nur für JSON im Fach Prüfen.
 - Die Partie im Fluss-Fach starten (sonst zwei Runtime)
 - Undo-Stack über mehrere Ebenen (nur eine Stufe zurück)
-

Ergänzung: Anfassen direkt auf der Bühne

Fach Karte deckt inhaltlich schon alles ab – Titel, Text, Bild, Portrait, Wahlen, Diff, Rückgängig. Was fehlt, ist nur die Art, wie

man hinkommt: man öffnet die Schublade, sucht das richtige Feld im Formular. Der ursprüngliche Wunsch war direkter: auf die Sache zeigen, die nicht passt, und sie genau dort ändern, ohne den Umweg über ein Formular.

Das ist keine zweite Speicherlogik – `welt.ts` mit `merkeAuflage()`, `auflageFuerSicht()`, `kanonDiff()`, `rueckgaengigAuflage()` bleibt exakt wie es ist. Es ist nur eine zweite Tür zu denselben Funktionen, direkt auf der Bühne statt im Fach Karte.

Wie es sich anfühlt

Du spielst. Du drückst `Alt+S` wie bisher – aber statt dass sich nur die Schublade öffnet, bekommt jedes sichtbare Element auf der Bühne einen leisen Umriss, sobald die Maus darüberfährt. Ein Klick auf den Titel öffnet ein kleines Feld genau an der Stelle, an der der Titel steht. Ein Klick auf das Hintergrundbild zeigt eine kleine Bildauswahl darüber. Die Schublade bleibt für alles, was einen Überblick braucht – Diff, Rückgängig, die Effekt-Chips –, aber die alltägliche Änderung eines einzelnen Satzes braucht sie nicht mehr.

Die Grundlage: eine Adresse pro sichtbarem Feld

```
// + neue Datei: src/game/anfassen.ts
export type AnfassAdresse =
  | { feld: "titel" }
  | { feld: "zeile"; index: number }
  | { feld: "wahl"; index: number }
  | { feld: "hintergrund" }
  | { feld: "portrait" };

export function adresseZuText(a: AnfassAdresse):
string {
  switch (a.feld) {
```

```

    case "titel": return "Titel";
    case "zeile": return `Zeile ${a.index + 1}`;
    case "wahl": return `Wahl ${a.index + 1}`;
    case "hintergrund": return "Hintergrundbild";
    case "portrait": return "Portrait";
  }
}

```

Diese Adresse braucht keine neue Speicherform — sie beschreibt nur, **welcher Teil** der bereits vorhandenen `WeltAuflage` (aus `welt.ts`) gerade gemeint ist. Ein Klick auf `{ feld: "titel" }` bedeutet am Ende schlicht `onChange({ ...auflage, title: neuerText })` — derselbe Aufruf, den `WeltKarte.tsx` heute schon für das Titel-Eingabefeld macht.

Der Zustand, der Hover und Auswahl trägt

```

// + neue Datei: src/game/anfassen-context.tsx
import { createContext, useContext, useState, type
  ReactNode } from "react";
import type { AnfassAdresse } from "../anfassen";

type Ctx = {
  gewaehlt: AnfassAdresse | null;
  waehle: (a: AnfassAdresse | null) => void;
};

const AnfassenCtx = createContext<Ctx | null>(
  null);

export function AnfassenProvider({ children }: {
  children: ReactNode }) {
  const [gewaehlt, setGewaehlt] =
    useState<AnfassAdresse | null>(null);
  return <AnfassenCtx.Provider value={{ gewaehlt,
    waehle: setGewaehlt }}>{children}
  </AnfassenCtx.Provider>;
}

```

```

}

export function useAnfassen() {
  const ctx = useContext(AnfassenCtx);
  if (!ctx) throw new Error("useAnfassen ohne
AnfassenProvider");
  return ctx;
}

```

Ob der Modus überhaupt aktiv ist, entscheidet weiterhin `weltAktiv()` aus `welt.ts` — kein zweiter Schalter, dieselbe Taste `Alt+S`, dasselbe Flag.

Der Wrapper um jedes anfassbare Element

```

// + neue Datei: src/components/game/Anfassbar.tsx
import { autoUpdate, offset, useFloating } from
"@floating-ui/react"; // + npm install @floating-
ui/react (MIT, ~5 kB)
import { type ReactNode, useState } from "react";
import { useAnfassen } from "@game/anfassen-
context";
import type { AnfassAdresse } from
"@game/anfassen";
import { adresseZuText } from "@game/anfassen";

export function Anfassbar({
  adresse,
  aktiv,
  children,
  editor,
}: {
  adresse: AnfassAdresse;
  aktiv: boolean; // = weltAktiv()
  children: ReactNode;
  editor: (schliessen: () => void) => ReactNode;
}) {

```

```

const { gewaehlt, waehle } = useAnfassen();
const [hover, setHover] = useState(false);
const offen = gewaehlt === adresse;
const { refs, floatingStyles } = useFloating({
  open: offen,
  placement: "bottom-start",
  middleware: [offset(6)],
  whileElementsMounted: autoUpdate,
});

if (!aktiv) return <>{children}</>;

return (
  <span
    ref={refs.setReference}
    onMouseEnter={() => setHover(true)}
    onMouseLeave={() => setHover(false)}
    onClick={(e) => {
      e.stopPropagation();
      waehle(offen ? null : adresse);
    }}
    title={adresseZuText(adresse)}
    className={
      "cursor-pointer rounded outline outline-2
transition-colors " +
      (offen ? "outline-amber-400" : hover ?
"outline-amber-300/50" : "outline-transparent")
    }
  >
    {children}
    {offen && (
      <span ref={refs.setFloating} style=
{floatingStyles} className="z-50">
        {editor(() => waehle(null))}
      </span>
    )}
  </span>
)

```

```
);
}
```

Einbau in `SceneStage.tsx` – bestehende Funktionen wiederverwenden

`SceneStage.tsx` bekommt bereits `patch: KartePatch` und `onPatch` als Props (siehe Werkstatt-Stand). Der Anfassen-Wrapper ruft genau `onPatch` auf, nichts Neues:

```
// + Titel-Zeile umschließen
<Anfassbar
  adresse={{ feld: "titel" }}
  aktiv={weltAktiv()}
  editor={(schliessen) => (
    <TextFeld
      wert={patch.title ?? view.title}
      aufSpeichern={(neu) => { onPatch({ ...patch,
title: neu }); schliessen(); }}
      aufAbbrechen={schliessen}
    />
  )}
>
  <h2>{view.title}</h2>
</Anfassbar>
```

```
// + Jede Zeile einzeln, jede Wahl einzeln –
gleiches Prinzip:
{view.lines.map((line, i) => (
  <Anfassbar key={i} adresse={{ feld: "zeile",
index: i }} aktiv={weltAktiv()} editor={/*
Textarea-Popover, schreibt in patch.lines[i] */}>
    <p>{line}</p>
  </Anfassbar>
)}})
```

```
// + Hintergrund – zeigt beim Klick eine
// Miniaturauswahl aus ART, bereits importiert in
// SceneStage.tsx:
<Anfassbar
  adresse={{ feld: "hintergrund" }}
  aktiv={weltAktiv()}
  editor={(schliessen) => (
    <BildRaster
      quelle={ART}
      aktuell={patch.art ?? view.art}
      aufWahl={art) => { onPatch({ ...patch, art
    }}); schliessen(); }}
    />
  )}
>
  <img src={artSrcFor(view.art)} alt="" />
</Anfassbar>
```

`TextFeld` und `BildRaster` sind zwei kleine, neue, generische Komponenten – kein neues Muster, nur ausgelagert, weil beide an mehreren Stellen gebraucht werden. `onReset`, `onRueckgaengig` und die Diff-Ansicht aus `WeltKarte.tsx` bleiben unverändert in der Schublade; sie werden von der Bühne aus nicht dupliziert.

Was mit dieser Ergänzung bewusst nicht mitkommt

- Die Effekt-Chips (was eine Karte gibt/nimmt) bleiben in Fach Karte – das sind Zustände, keine sichtbaren Bühnenelemente, für die ein Klick-auf-die-Bühne keinen Sinn ergibt.
- Eine neue Seite anlegen bleibt vorerst der Werkstatt vorbehalten (`WeltEntwurf.tsx`, bereits vorhanden) – kein Versuch, zur Laufzeit Code in `script.ts` zu schreiben.

- Kein zweiter Speicher, keine zweite Zod-Struktur – `WeltAuflage` bleibt die einzige Form, in der eine Änderung existiert.

Reihenfolge, wenn gebaut wird (nicht jetzt)

#	Schritt	Risiko
0	Dieser Plan. Oberfläche unangetastet.	—
1	<code>id an present</code> , in <code>SceneView</code>	niedrig
2	<code>welt.v2</code> + einmalige Übernahme der zwei Alt-Speicher	niedrig
3	Eine Schublade, drei Fächer, HUD Welt , Titel ein Einstieg	mittel (nur UI)
4	HUD-Badges (Wissen-Zahl, Welt-Punkt) + Rückgängig auf Auflage	niedrig, nach 3
5	<code>/editor</code> und Textmodus-Flag entfernen	niedrig, nach 3
6	Kanon-Diff vor dem Schreiben	niedrig, nach 3
7	Kanon-Schreiben nur für Module, die schon <code>content.ts</code> sind	mittel, einzeln
8	Schwierigkeitskurve + Export pro Modul in Fach Prüfen	niedrig, nach 7
9	Anfassen direkt auf der Bühne (Hover, Klick, Inline-Editor über	niedrig, reine Erweiterung von

#	Schritt	Risiko
	Anfassbar)	Fach Karte

Spielbar nach jedem Schritt. Kein Big-Bang.

Erfolg

Du spielst. Du öffnest **Welt**. Du siehst am Knopf, dass diese Karte schon einmal geändert wurde. Du änderst den Satz am Brunnen. Du siehst **gemerkt (Auflage)**. Bei Bedarf eine Stufe zurück. Die Partie speicherst du extra. In den Kanon nur, wenn du das willst – und du siehst vorher die Zeilen. Die Werkstatt ist kein zweites Spiel, sondern Fach **Prüfen**.